Topic 4: Files

Input and Output Files

Learning Outcomes:

- Identify the use of input/output files using text and CSV files
- Open, read, write, and close `.txt` files
- Open, read, write, and close `.csv` files
- Apply modular design when working with files

Prerequisites: Topic 1 (Functions), Topic 2 (Lists)

Opening Problem

You're building a student grade tracker.

The program calculates and stores student scores during runtime.

```
scores = []
scores.append(85)
scores.append(92)
scores.append(78)

print("Scores:", scores)
```

Output:

```
Scores: [85, 92, 78]
```

What happens when the program ends?

The Problem: Data Does Not Persist

```
scores = []
scores.append(85)
scores.append(92)
scores.append(78)

# Program ends here
# All data in memory is erased
```

Next time the program runs:

```
print(scores)
# []  — all previous data is gone
```

Every run starts from scratch. Data lives only in memory.

There must be a better way.

The Solution: Files

Files store data on disk — data persists after the program ends.

```
f = open("scores.txt", "w")
f.write("85\n")
f.write("92\n")
f.write("78\n")
f.close()
```

Next time the program runs:

```
f = open("scores.txt", "r")
data = f.read()
print(data)
f.close()
```

Output:

Why Use Files?

Reason	Explanation
Data Persistence	Data stored in memory is deleted when program ends
Data Exchange	Common format for sharing data between programs
File Handling Skills	Essential operations for real-world applications
Configuration	Applications use files to store settings
Data Processing	Files serve as input sources for processing tasks
Compatibility	Platform-independent and transferable between systems

Two Types of Files in Python

Type	Description	Examples
Text File *(our focus)*	Stores human-readable character data	`.txt`, `.csv`, `.json`, `.xml`, `.yaml`
Binary File	Stores binary data	Images, audio, video files

This topic focuses on text files — specifically `.txt` and `.csv`.

What is a Text File?

A text file:

- Contains human-readable characters
- No embedded formatting commands (no bold, no italic)
- Data represented as sequences of ASCII characters organized into lines
- Lines are typically terminated by a newline character (`\n`)

Common text file formats:

```
.txt    .csv    .json    .xml    .yaml
```

.txt vs .csv

Feature	.txt	.csv
Content	Unformatted plain text	Structured tabular data
Structure	Free-form lines	Rows and columns, comma-separated
Open with	Any text editor	Excel, Google Sheets, text editor
Use case	Logs, notes, configs	Records, datasets, exports

Structure of a .csv File

```
Name,Score,Grade
Ali,85,A
Sara,92,A+
Ahmad,78,B
```

Breaking it down:

Component	Description
Header	First row — describes each column (`Name` , `Score` , `Grade`)
Record	Each subsequent row — one entry of data
Field	Each value within a record, separated by commas

CSV = Comma Separated Values

Exercise 1: Identify File Components

Given this CSV content:

```
ProductID,Name,Price
101,Laptop,2500
102,Mouse,45
103,Keyboard,120
```

Identify:

1. What is the header?
2. How many records are there?
3. What are the fields of the second record?

File Operations: The 4 Steps

Every file interaction follows the same 4 steps:

```
1. Open   →   2. Read or Write   →   3. (Process)   →   4. Close
```

Python provides built-in functions for `.txt` and an external module for `.csv`:

File Type	Module Required
`.txt`	None — built-in
`.csv`	`import csv`

Structure of `open()`

```
f = open("demofile.txt", "r")
```

Breaking it down:

```
function name    file path        mode
     │               │             │
     ▼               ▼             ▼
   open(    "demofile.txt",       "r"    )
              │
              └── filename in same folder, or full path
```

Component	Description
`open`	Built-in function to access a file
`"demofile.txt"`	Path to the file

Opening Modes

Mode	Name	Action	Behaviour
`r`	Read	Read only	Error if file does not exist
`w`	Write	Write only	Creates file if not exists; **overwrites** if exists
`a`	Append	Write to end	Creates file if not exists; appends if exists
`r+`	Read/Write	Read and write	Error if file does not exist
`w+`	Write/Read	Write and read	Creates file if not exists; **overwrites** if exists
`a+`	Append/Read	Append and read	Creates file if not exists; appends if exists

Understanding the `+` Modifier

Adding `+` to a mode allows the opposite operation as well:

```
r   →  read only              r+  →  read AND write
w   →  write only             w+  →  write AND read
a   →  append only            a+  →  append AND read
```

In practice — `w+` is commonly used with CSV:

```
import csv

# w+ : create the file and write to it
# The + means you can also read it back later in the same program
f = open("scores.csv", "w+", newline="")
writer = csv.writer(f)
writer.writerow(["Name", "Score"])
writer.writerow(["Ali", 85])
f.close()
```

Opening File Syntax in Different Locations

File Location	Syntax
Same folder	`open('demo.txt', 'r')`
Local path	`open('c:/Users/user/Documents/project/demo.txt', 'r')`
Google Drive (Colab)	`open('/content/drive/MyDrive/project/demo.txt', 'r')`

Use forward slashes `/` in file paths, even on Windows.

Working with .txt Files

.txt Python File Operation Functions & Methods:

```
# Open
f = open(path, mode)

# Read
f.read()       # Read entire content as a STRING
f.readline()   # Read a single line as a STRING
f.readlines()  # Read all lines, return a LIST of STRINGs

# Write
f.write()      # Write a single string to the file
f.writelines() # Write a list of strings to the file

# Close
f.close()      # Close the file
```

.txt — Step 1: Open

Syntax:

```
f = open("demofile.txt", "r")
```

File does not exist — `FileNotFoundError`:

```
f = open("demofiles.txt", "r")
```

Output:

```
FileNotFoundError: [Errno 2] No such file or directory: 'demofiles.txt'
```

Error always occurs when either the directory is wrong or the file does not exist.

.txt — Step 2(a): Read Entire Content

`f.read()` — reads the entire file as a single string:

```
f = open("demofile.txt", "r")
text = f.read()
print(text)
print(type(text))
f.close()
```

Input file (`demofile.txt`):

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```

Output:

```
Hello! Welcome to demofile.txt
```

.txt — Step 2(b): Read a Single Line

`f.readline()` — reads one line each time it is called:

Called once:

```
f = open("demofile.txt", "r")
line = f.readline()
print(line)
f.close()
```

Output:

```
Hello! Welcome to demofile.txt
```

.txt — Step 2(b): Read a Single Line (cont')

Called twice — reads first and second line:

```
f = open("demofile.txt", "r")
line1 = f.readline()
line2 = f.readline()
print(line1)
print(line2)
f.close()
```

Output:

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
```

Each call to `readline()` advances to the next line.

.txt — Step 2(c): Read All Lines

`f.readlines()` — reads all lines and returns a list of strings:

```
f = open("demofile.txt", "r")
lines = f.readlines()
print(lines)
f.close()
```

Output:

```
['Hello! Welcome to demofile.txt\n', 'This file is for testing purposes.\n', 'Good Luck!\n']
```

Each element in the list is one line. The `\n` character is included.

Read Functions: Return Type Summary

Function	Returns	Use When
`read()`	Single `str` (entire file)	Processing entire file as text
`readline()`	Single `str` (one line)	Processing one line at a time
`readlines()`	`list` of `str`	Processing each line individually

Note: All return values are strings. Convert as needed.

```
line = f.readline()
number = int(line.strip())   # Convert string to int
```

Exercise 2: Predict the Output

Given `data.txt`:

```
Python
Files
Topic 4
```

What does each print?

```python
# Code A
f = open("data.txt", "r")
print(f.readline())
print(f.readline())
f.close()

# Code B
f = open("data.txt", "r")
print(f.readlines())
f.close()
```

.txt — Step 3: Write File Syntax

`f.write()` **writes a specified string to the file.**

Behaviour depends on mode:

- `"w"` — empties the file first, then writes from position 0
- `"a"` — keeps existing content, writes at the end

```
f = open("expenses.txt", "w")
f.write("content to write\n")
f.close()
```

Or equivalently:

```
f.write("content to write" + "\n")
```

`\n` **is added to the end to ensure the next string starts on a new line.**

.txt — Step 3(a): Write a Single String

```
f = open("scores.txt", "w")
f.write("Ali: 85\n")
f.close()
```

File content (`scores.txt`):

```
Ali: 85
```

Writing a list using `write()` inside a for loop:

```
names = ["Ali\n", "Sara\n", "Ahmad\n"]

f = open("names.txt", "w")
for name in names:
    f.write(name)
f.close()
```

.txt — Step 3(b): Write List of Strings

`f.writelines()` — a more concise way to write a list:

```
names = ["Ali\n", "Sara\n", "Ahmad\n"]

f = open("names.txt", "w")
f.writelines(names)
f.close()
```

File content (`names.txt`):

```
Ali
Sara
Ahmad
```

`writelines()` does not add `\n` automatically — each string must include it.

Write Methods: Comparison

Method	Input	Adds `\n`?	Use When
`write(str)`	Single string	No	Writing one line at a time
`write(str)` in loop	Loop over list	No	Writing each element individually
`writelines(list)`	List of strings	No	Writing all lines at once (concise)

Exercise 3: Write to File

Write code to:

1. Open `"students.txt"` in write mode
2. Write the following names, each on a new line:
    - "Fatimah"
    - "Hafiz"
    - "Nurul"
3. Close the file

Then write a second version using `writelines()`.

.txt — Step 4: Close File

```
f.close()
```

Why closing matters:

- Releases the file from the program's control
- Ensures all written data is saved (flushed) to disk
- Allows other programs to access the file
- Prevents resource leaks — open files consume system resources

Always call `f.close()` after you are done with the file.

.txt: Complete Example

A complete read → process → write workflow:

```
def read_scores(filename):
    f = open(filename, "r")
    lines = f.readlines()
    f.close()
    return lines

def write_scores(filename, scores):
    f = open(filename, "w")
    for score in scores:
        f.write(score)
    f.close()

scores = read_scores("scores.txt")
write_scores("scores_backup.txt", scores)
```

Each operation is isolated in its own function.

Working with .csv Files

.csv requires importing the `csv` module:

```
import csv
```

.csv Python File Operation Functions & Methods:

```
# Open
f = open(path, mode, newline="")

# Read
reader = csv.reader(f)        # Create reader object
# Iterate reader to read content row by row

# Write
writer = csv.writer(f)
writer.writerow([...])        # Write a single row
writer.writerows([[...]])     # Write multiple rows
```

.csv — Step 1: Open

Syntax:

```
import csv
f = open("expenses.csv", "r", newline="")
```

Breaking it down:

Part	Description
`"expenses.csv"`	Path to the CSV file
`"r"`	Mode — same modes as `.txt`
`newline=""`	Prevents extra blank lines between rows

Same file location rules apply as `.txt`.

Opening File Syntax in Different Locations (.csv)

File Location	Syntax
Same folder	`open('demo.csv', 'w+', newline="")`
Local path	`open('c:/Users/user/Documents/demo.csv', 'w+', newline="")`
Google Drive (Colab)	`open('/content/drive/MyDrive/demo.csv', 'w+', newline="")`

.csv — Step 2: Read File Syntax

Reading a `.csv` file requires a `reader` object:

```
import csv
f = open("expenses.csv", "r", newline="")
reader = csv.reader(f)
```

The `reader` object:

- Reads the entire content of the file
- Must be iterated to access the content row by row

.csv — Step 2(a): Read Content of File

Iterate over `reader` to print each row:

```
import csv

f = open("daily_expenses.csv", "r", newline="")
reader = csv.reader(f)

for row in reader:
    print(row)

f.close()
```

Input file (`daily_expenses.csv`):

```
Date,Item,Amount
2024-01-01,Food,15.50
2024-01-02,Transport,5.00
```

.csv — Step 2(b): Save Content into a List

Use `list.extend()` to save all rows into a list for further processing:

```
import csv

f = open("daily_expenses.csv", "r", newline="")
reader = csv.reader(f)

content = []
for row in reader:
    content.extend([row])

f.close()
print(content)
```

Output:

```
[['Date', 'Item', 'Amount'], ['2024-01-01', 'Food', '15.50'], ['2024-01-02', 'Transport', '5.00']]
```

Exercise 4: Read a CSV

Given `students.csv`:

```
Name,Score
Ali,85
Sara,92
Ahmad,78
```

Write code to:

1. Open the file in read mode with `newline=""`
2. Create a `reader` object
3. Print each row
4. Close the file

.csv — Step 3: Write File Syntax

Writing a `.csv` file requires a `writer` object:

```
import csv
f = open("expenses.csv", "w+", newline="")
writer = csv.writer(f)
```

Two write methods:

Method	Purpose
`writer.writerow([field1, field2, ...])`	Write a single row
`writer.writerows([[row1], [row2], ...])`	Write multiple rows at once

.csv — Step 3(a): Open with `w+` and `writerow()`

`w+` mode — creates the file if it does not exist; overwrites if it does:

```
import csv

f = open("scores.csv", "w+", newline="")
writer = csv.writer(f)

writer.writerow(["Name", "Score"])
writer.writerow(["Ali", 85])
writer.writerow(["Sara", 92])

f.close()
```

File content (`scores.csv`):

```
Name,Score
Ali,85
Sara,92
```

.csv — Step 3(a): Using `writerow()` in a Loop

Write a list of records using `writerow()` inside a for loop:

```
import csv

records = [["Ali", 85], ["Sara", 92], ["Ahmad", 78]]

f = open("scores.csv", "a+", newline="")
writer = csv.writer(f)

for record in records:
    writer.writerow(record)

f.close()
```

`a+` mode — appends new records to existing content.

.csv — Step 3(b): Open with `a+` and `writerows()`

`writerows()` — a more concise way to write multiple rows at once:

```
import csv

records = [
    ["Ali", 85],
    ["Sara", 92],
    ["Ahmad", 78]
]

f = open("scores.csv", "a+", newline="")
writer = csv.writer(f)
writer.writerows(records)
f.close()
```

Opening with `a+` mode — new content is appended to existing content.

Exercise 5: Write to CSV

Write code to create `products.csv` with the following content:

```
ID,Name,Price
101,Laptop,2500
102,Mouse,45
103,Keyboard,120
```

Requirements:

1. Use `writerow()` for the header
2. Use `writerows()` for the data rows
3. Close the file after writing

.csv — Step 4: Close File

```
f.close()
```

Same as `.txt` — always close after processing.

.csv: Complete Example

A complete read → process → write workflow using functions:

```
import csv

def read_csv(filename):
    f = open(filename, "r", newline="")
    reader = csv.reader(f)
    content = []
    for row in reader:
        content.extend([row])
    f.close()
    return content

def write_csv(filename, records):
    f = open(filename, "w+", newline="")
    writer = csv.writer(f)
    writer.writerows(records)
    f.close()
```

The Principled Approach

6 principles for working with files correctly:

#	Principle	The Rule
1	Validate File Existence	Create the file with `"w"` before reading it with `"r"`
2	Choose the Right Mode	`"w"` destroys content. `"a"` preserves it. Choose deliberately
3	Modular Approach	Wrap file operations in functions — one function per responsibility
4	Include `\n` When Writing	Without it, lines merge. Each written string must end with `\n`
5	Always Close Files	Every `open()` must have a matching `f.close()`

Principle 1: Validate File Existence

The Problem:

```
f = open("scores.txt", "r")   # File does not exist yet
```

Output:

```
FileNotFoundError: [Errno 2] No such file or directory: 'scores.txt'
```

The Solution:

```
# Create the file first with "w" or "w+"
f = open("scores.txt", "w")
f.write("85\n")
f.close()

# Now it exists — safe to read
f = open("scores.txt", "r")
```

Principle 2: Choose the Right Mode

The Problem:

```
# You want to add new scores — but use "w" by mistake
f = open("scores.txt", "w")
f.write("100\n")
f.close()
```

Result: All previous scores are gone. `"w"` overwrites the entire file.

The Solution:

```
# Use "a" to append — preserves existing content
f = open("scores.txt", "a")
f.write("100\n")
f.close()
```

The Rule: Choose the mode deliberately. `"w"` destroys existing content. `"a"` preserves

Principle 3: Modular Approach

The Problem:

```
# File logic repeated throughout the program
f = open("scores.txt", "r")
lines = f.readlines()
f.close()

# ... elsewhere in the same program ...
f = open("scores.txt", "r")
lines = f.readlines()
f.close()
```

The Solution:

```
def read_scores(filename):
    f = open(filename, "r")
    lines = f.readlines()
    f.close()
```

Principle 3: Modular Approach (CSV)

The same principle applies to `.csv`:

```
import csv

def read_csv(filename):
    f = open(filename, "r", newline="")
    reader = csv.reader(f)
    content = []
    for row in reader:
        content.extend([row])
    f.close()
    return content

def write_csv(filename, records):
    f = open(filename, "w+", newline="")
    writer = csv.writer(f)
    writer.writerows(records)
    f.close()
```

Principle 4: Include `\n` When Writing

The Problem:

```
f = open("scores.txt", "w")
f.write("Ali: 85")
f.write("Sara: 92")
f.write("Ahmad: 78")
f.close()
```

File content:

```
Ali: 85Sara: 92Ahmad: 78
```

All entries are merged into one line.

The Solution:

```
f = open("scores.txt", "w")
```

Principle 5: Always Close Files

The Problem:

```
f = open("scores.txt", "w")
f.write("Ali: 85\n")
# Forgot to close — data may not be saved to disk
```

The Solution:

```
f = open("scores.txt", "w")
f.write("Ali: 85\n")
f.close()   # Always close
```

The Rule: Every `open()` must have a matching `f.close()`. Unclosed files risk data loss and resource leaks.

Principle 6: Use `newline=""` for CSV

The Problem:

```
f = open("expenses.csv", "w")   # Missing newline=""
writer = csv.writer(f)
writer.writerow(["Ali", 85])
writer.writerow(["Sara", 92])
f.close()
```

File content:

```
Ali,85
          ← extra blank line
Sara,92
          ← extra blank line
```

The Solution:

Common Mistakes

1. Reading a file that has not been created yet:

```
f = open("scores.txt", "r")   # FileNotFoundError
```

Use `"w"` or `"w+"` to create the file first.

2. Wrong mode overwrites data:

```
f = open("scores.txt", "w")   # Destroys existing content
```

Use `"a"` to append, not `"w"`.

3. Missing `\n` when writing:

```
f.write("Ali")
f.write("Sara")   # AliSara — merged into one line
```

Common Mistakes (Continued)

4. Missing `import csv` for CSV files:

```
reader = csv.reader(f)   # NameError: csv is not defined
```

Always add `import csv` at the top of the file.

5. Missing `newline=""` for CSV:

```
f = open("data.csv", "w")   # Extra blank lines between rows
```

Always use `open("data.csv", "w", newline="")`.

6. Not closing the file:

```
f = open("data.txt", "w")
f.write("hello\n")
# File not closed — data may not be saved
```

Summary

What we covered:

1. **Why files?** — Data persistence, exchange, processing
2. **File types** — Text (`.txt` , `.csv`) vs Binary
3. **.txt operations** — `open()` , `read()` , `readline()` , `readlines()` , `write()` , `writelines()` , `close()`
4. **.csv operations** — `import csv` , `csv.reader()` , `csv.writer()` , `writerow()` , `writerows()` , `close()`
5. **Modes** — `r` , `w` , `a` , `r+` , `w+` , `a+` — choose deliberately
6. **6 Principles** — Validate existence, right mode, modular approach, `\n` , close files, `newline=""` for CSV

Principles Quick Reference

#	Principle	Bad	Good
1	Validate Existence	Read without writing first	Write first, then read
2	Right Mode	`"w"` when appending	`"a"` to preserve content
3	Modular Approach	Logic scattered in program	Wrap in dedicated functions
4	Include `\n`	`write("data")`	`write("data\n")`
5	Always Close	No `f.close()`	Always call `f.close()`
6	`newline=""` for CSV	`open("f.csv", "w")`	`open("f.csv", "w", newline="")`

What's Next

Data stored in files is often processed with the data structures we already know:

```
import csv

def load_scores(filename):
    f = open(filename, "r", newline="")
    reader = csv.reader(f)
    scores = []
    for row in reader:
        scores.append(row)
    f.close()
    return scores

data = load_scores("scores.csv")
print(data[0])   # First record — a list
```

Files + Lists + Functions = a complete data processing pipeline.